User's Manual for ASRgwas v. 1.0.0

An R package to perform complex Genome-Wide Association Studies (GWAS)

Prepared by

Giovanni Galli, Salvador A. Gezan,
Didier A. Murillo and Darren Murray

November 17, 2022

User's Manual for ASRgwas v. 1.0.0

Bibliographical reference

Galli, G., Gezan, S.A., Murillo, D.A., and Murray, D. 2022. ASRgwas: An R package to perform complex Genome-Wide Association Studies (GWAS). Version 1.0.0. VSN International, Hemel Hempstead, United Kingdom.

Authors' email addresses

Giovanni Galli giovanni.galli@vsni.co.uk
Salvador A. Gezan salvador.gezan@vsni.co.uk
Didier A. Murillo didier.murilloflorez@ndsu.edu
Darren Murray darren.murray@vsni.co.uk

Companion resources

The R library and associated documentation can be downloaded from:
https://vsni.co.uk/free-software/asrgwas

Acknowledgements

The authors acknowledge the contributions for this library from Dr. Humberto Fanelli Carvalho with its help for the initial code formulation of this library. The following reviewers helped with important comments and suggestions: Dang Fei (VSNi), Khaled Al-Sham'aa (ICARDA-Egypt) and Dr. Rafael Massahiro Yassue (GDM Seeds). Finally, Dr. Vanessa Cave (VSNi) who assisted us enormously with editorial comments.

Preface

`ASRgwas` is a comprehensive R package designed to perform Genome-Wide Association Studies (GWAS) using the modelling flexibility available in `ASReml-R` in an efficient and reliable way. This library assists with preparing data and matrices, and verifies that they are adequate to perform GWAS. In addition, it has a set of complementary functions to be used for *post*-GWAS analyses to help with interpretation, use of the output information, and obtaining graphical outputs.

The main tasks considered within `ASRgwas` are:

- Preparing and auditing phenotypic and genomic data.
- Fitting GWAS models and identifying significant markers.
- Evaluating results and generating tables and graphical output.

The intent of this tool is to facilitate the execution of GWAS in a straightforward and efficient manner, along with providing full reproducibility of these analyses. We have used state-of-the-art approaches and algorithms to obtain reliable and fast estimation of marker effects. In addition, we have made use of parallelization, whenever possible, and fast matrix operation routines (*e.g.* C++) available within the software `R`.

`ASRgwas` is designed to allow for any number of fixed and/or random structures, and heterogenoeus error variances. It also accepts raw and replicated data. All of this allows for better use of the available phenotypic information and full use of the linear mixed model (LMM) methodology as available in `ASReml-R` (Butler et al. 2009). In addition, this package accepts missing values in the marker information avoiding the need to implement marker imputation.

As part of `ASRgwas` we have extended the analytical options within GWAS by allowing the evaluation of phenotypic responses that follow a Binomial distribution. This extension into generalized linear mixed models (GLMMs), a wider class of models with random effects for non-Normal errors, also uses `ASReml-R`.

In this manual, we will present some of the logic and options behind `ASRgwas`, illustrating its capabilities with examples using a range of datasets. This package is a tool that is provided "as is". We have made all efforts to check our routines carefully and we have tested them with an array of datasets. But let us know if you find issues or if you would like extensions to these analyses.

Contents

1 Overview of Genome-Wide Association Studies

Association studies have been around for a few decades in plant and animal breeding. In a GWAS study, the whole genome is studied for genomic variants associated with a particular trait. These methods rely on the correlation that exists between a genetic marker and a phenotype among a collection of germplasm. The idea is to detect specific markers that can be used to identify a given differential response on a group of genotypes evaluated. These markers are key to tools such as **Marker Assisted Selection** (MAS); its use can result in important genetic gains by focusing on only a small set of molecular markers in a large pool of individuals.

The simplest approach to find those *significant* markers is based on a t-test that evaluates each marker individually. However, phenotypic data has a complex structure requiring, for example, the inclusion of design factors. At present, most GWAS applications use linear mixed models (LMMs) to fit complex models controlling for different data aspects and therefore providing more accurate results. Below, we introduce the standard model and procedures in GWAS used for plant, animal, aquaculture, and even human genetics to detect these *significant* markers on a wide range of response variables.

The general model used for GWAS is defined below:

$$\mathbf{y} = \mathbf{1}\mu + \alpha_i \mathbf{s}_i + \mathbf{X}\boldsymbol{\beta} + \mathbf{Z_1 u} + \mathbf{Q}\boldsymbol{\delta} + \mathbf{Z_2 a} + \mathbf{e}$$

where

μ is the overall mean,
α_i is the slope associated with the ith marker,
$\boldsymbol{\beta}$ is the vector of fixed design effects,
$\mathbf{u}$ is the vector of random design effects (*i.e.*, blocks, pens), with $\mathbf{u} \sim \mathbf{N}(\mathbf{0}, \mathbf{I}\sigma^2_{\mathbf{u}})$,
$\boldsymbol{\delta}$ is the vector of fixed group or population effects,
$\mathbf{a}$ is the vector of random additive effects (*i.e.*, breeding values), with $\mathbf{a} \sim \mathbf{N}(\mathbf{0}, \mathbf{G_A}\sigma^2_{\mathbf{A}})$,
$\mathbf{e}$ is the vector of random residual effects, with $\mathbf{e} \sim \mathbf{N}(\mathbf{0}, \mathbf{I}\sigma^2)$,
$\mathbf{s}_i$ is the vector of values (0, 1, 2) for the ith marker extracted from the marker matrix $\mathbf{M}$.

The matrices $\mathbf{X}$, $\mathbf{Z_1}$ and $\mathbf{Z_2}$ are incidence matrices, $\mathbf{1}$ is a vector of ones, $\mathbf{Q}$ is a matrix of q vectors describing the population structure, and finally $\mathbf{G_A}$ is the genomic relationship matrix (GRM) derived from the markers.

As noted above, this model has several fixed and random effects that play an important role in GWAS models. We will describe each of these terms below.

1. **Marker Effects**. Estimating marker effects is the main objective of a GWAS analysis. The estimate associated with α_i is the slope of the ith marker, and the data for the associated explanatory variable comes from the ith column of the marker matrix $\mathbf{M}$ (*i.e.*, $\mathbf{s}_i$).

2. **Design Model Terms**. These include both fixed (*i.e.*, $\boldsymbol{\beta}$) and random effects (*i.e.*, $\mathbf{u}$) associated with the data. Often, in plant breeding, replicates and incomplete blocks are included here, or for animal/aquaculture we have herd-year-season or pens. In addition, covariates can be also considered.

3. **Population Structure**. The presence of some genetic structure in the population needs to be controlled for. This is usually done by considering the first few eigenvectors from the marker data matrix $\mathbf{M}$, or from the genomic relationship matrix $\mathbf{G_A}$, to form the matrix of coefficients $\mathbf{Q}$ that is incorporated in the model as shown before. The decision on the number of dimensions to consider is often supported by a scree-plot.

4. **Family Structure**. This is another potential source of bias, which is controlled by incorporating the genomic relationship matrix $\mathbf{G_A}$ into the model. This matrix is part of the assumptions of the additive effects $\mathbf{a}$, and it is obtained directly from the marker data matrix $\mathbf{M}$ using different expressions, where the most common is the one proposed by VanRaden (2008).

5. **Residual Term**. Typically this is assumed to be independent and identically distributed, but depending on the complexity of the LMM specified, this can have many different error structures (*e.g.*, heteroscedastic errors).

There are many variants to the above general GWAS model. It is not uncommon to ignore some of the terms (such as the $\mathbf{Q}$ matrix), or to change the parameterization of the $\mathbf{M}$ matrix, that at present is coded for each marker in an additive way (0, 1, and 2) but can be coded as a dominance matrix (0, 1, 0) or with each marker considered as a factor for non-additive modelling. Some of these variants will be discussed here, but for further details we refer to the scientific literature on this topic.

2 Analytical Workflow with ASRgwas

2.1 General Description

The package `ASRgwas` is very flexible, and in order to facilitate its use and logic we will separate the workflow into the following stages:

- Preparing and auditing phenotypic and genomic data with the function `pre.gwas()`.
- Fitting a GWAS model with the function `gwas.asreml()`.
- Selecting the final set of markers with the function `select.marker()`.
- Extracting and interpreting results with tables and/or graphical output.

We consider the first step as part of the *pre*-GWAS analyses. This is where the phenotypic and genotypic data are read, explored and prepared, and new matrices, such as $\mathbf{Q}$ and $\mathbf{G_A}$, are calculated. However, this stage includes several additional data cleaning and processing decisions, and it is possible that the process needs to be suspended whilst some of the data is verified internally or externally (*e.g.*, presence of duplicate genotypes). `ASRgwas` contains the function `pre.gwas()` to assist with this process. The use of this function is critical as it will make everything ready and smooth for downstream analyses. This pre-processing was separated into a different step in order to help understand, explore and correct the data before the final analysis is executed.

The function `gwas.asreml()` is the main routine to fit the GWAS model. This function will read the data frames generated in the *pre*-GWAS process (or by another procedure as this function is flexible enough to accept other pre-specified data frames) and proceed to fit the LMM using `ASReml-R` in the background. There are several analytical options and specifications that need to be considered that will be discussed later. The function `gwas.asreml()` will scan over all markers found in the filtered genomic matrix $\mathbf{M}$, and proceed to determine their significance for a given response variable. Once this process is finished, a list of significant markers will be identified (this will depend on the p-value threshold used and quality of the data).

We consider the above list of markers as a *candidate* list, as they might be highly collinear, and they need to be screened further as a full set. For this, the function `select.marker()` is used to perform a **backward selection** procedure that will start with the list of pre-selected markers, refit the LMM with all markers, and eliminate, one at the time (those that are above a certain p-value threshold). The process stops once all markers are accepted within the LMM.

Finally, the output from the selected markers is ready to be extracted and interpreted. This *post*-GWAS process is assisted by some graphical functions included within the library `ASRgwas` or as specific output from the other functions used previously.

Providing a marker matrix without missing data allows the evaluation of the markers using the fast implementation of the algorithm **Population Parameters Previously Determined (P3D)** where variance components are first estimated without any markers,

and then a specific definition of the residuals from this model are used to estimate marker effects and their significance using matrix operations. This method is the fastest, and is the default in `gwas.asreml()` for Normally distributed responses. However, if there are missing data in the marker matrix, then this matrix algebra needs to be adapted, where each marker is evaluated individually using an extended algorithm based on the same base model. This algorithm relies on dropping rows and columns from the variance-covariance matrix of responses and adjusting this matrix by approximating it. This method with missing marker data is slower, but it eliminates the need for imputing marker data providing more reliable results.

It is also possible within `gwas.asreml()` to fit an exact model for each marker (with or without missing data). For this, the option `P3D = FALSE` needs to be specified. Here, each marker will be fitted individually into a LMM (or GLMM) to obtain its respective variance components and significance level. This methodology does not use any approximation and will be considerably slower but it is exact! Also, it is the only available option within `gwas.asreml()` for binomially distributed data.

It is important to note that the presence of missing data in the marker matrix $\mathbf{M}$ will effectively result in all rows containing missing data to be dropped for all markers when backward selection is performed using the function `select.marker()`. This will lead to a smaller dataset being used to fit the LMM, and potentially large differences in the estimated marker effects compared to when the markers are individually fitted in the LMM. In addition, the function `select.marker()` fits markers as covariates and no regularization is performed (as occurs with ridge regression). Hence, this procedure will effectively eliminate highly correlated markers.

2.2 Preparing the Genomic Dataset

Preparing a genomic dataset, in this case the $\mathbf{M}$ matrix, is critical to ensure that all downstream analyses run properly. An initial step is to filter this molecular dataset, as it might contain a large amount of missing data and/or non-informative markers. One important filtering process is to remove markers below a minor allele frequency (MAF) threshold, which is often set at 0.05. In addition, eliminating markers and individuals with a large proportion of missing values is critical (20% missing or more is often used). Additional filtering should consider the level of heterozygosity and/or inbreeding. Filtering can be executed within `ASRgwas` using the function `pre.gwas()`, which has several options and specifications for filtering. All options available within the function `qc.filtering()` from the library `ASRgenomics` (Gezan et al. 2022b) are accessible here.

After this filtering, `pre.gwas()` will proceed to obtain the genomic relationship matrix, $\mathbf{G_A}$, where the available methods for additive relationships are `VanRaden` (VanRaden 2008) and `Yang` (Yang 2010). For dominance relationships the options are `Su` (Su et al. 2012) and `Vitezica` (Vitezica et al. 2013). The resulting matrix will be audited and its inverse will be obtained. If there are some issues in this matrix, these will be reported and some tune-up will be implemented (mainly some *blending*). Further details can be found in the `ASRgenomics` manual.

Finally, the population structure matrix $\mathbf{Q}$ will be generated using the filtered $\mathbf{M}$ or $\mathbf{G_A}$ matrix, according to the user's specifications and based on the methodology presented by Patterson et al. (2006). From this matrix a number of eigenvectors will be extracted and stored for later use.

3 Getting Started

`ASRgwas` is an R package available for Linux, Windows and MacOS that can be downloaded from https://vsni.co.uk/free-software/asrgwas

Download the appropriate version of `ASRgwas` for your operating system.

- For Windows, the download will be a .zip file.
- For Linux, the download will be a .tgz file.
- For Mac, the download will be a .tar.gz file.

Before installing `ASRgwas` you will need to install the following packages:

- `ASRgenomics`
- `ASRdata`
- `cli`
- `data.table`
- `ggplot2`
- `ggrepel`
- `Rcpp`
- `RcppArmadillo`

Most packages and their dependencies are available for installation from CRAN (https://cran.r-project.org/) via:

```
install.packages(c("cli", "data.table", "ggplot2",
    "ggrepel", "Rcpp", "RcppArmadillo"), dependencies = TRUE)
```

`ASRdata` can be installed via:

```
install.packages("devtools")
devtools::install_github("asrtools/asrdata")
```

`ASRgenomics` can be downloaded from https://vsni.co.uk/free-software/asrgenomics.

Install `ASRgwas` and `ASRgenomics` using one of the following commands as appropriate for your operating system.

For Windows:

```
install.packages(path, repos = NULL, type = "win.binary")
```

For Linux:

```
install.packages(path, repos = NULL)
```

For Mac:

```
install.packages(path, repos = NULL, type = "source")
```

where `path` is the location and name of the appropriate file for your operating system to install, for example: `"/home/<yourusername>/ASRgwas1.0.0.zip"`.

Another option is to install `ASRgwas` and `ASRgenomics` directly from `RStudio` by first going to the menu: `>Tools/InstallPackages...`, in `Install from` select

`"Package Archive File (.zip; .tgz; tar.gz)"`

and then you will have to search for the location of the file on your computer and select it. Finally, you just need to click on `Install`.

Once installed, load the library using the command:

```
library(ASRgwas)
```

Now you are ready to use it! A good way to get started is to request help directly from this library. For example, you can type:

```
help(ASRgwas)
```

This provides a complete description of the functions and the datasets used in this manual.

4 Traditional Genome-Wide Association Studies

In this section we will present how to prepare and run a GWAS model based on the generic description presented earlier. To illustrate the flow of functions with `ASRgwas` we are going to use a real dataset from Atlantic Salmon (*Salmo salar*) published by Robledo et al. (2018). The phenotypic data consists of a total of 1,481 individuals that were phenotyped for mean gill score and amoebic load (qPCR values). Also, all individuals were genotyped and a total of 17,156 SNP markers are available (coded as 0, 1, 2 and `NA`). There is no pedigree or marker map information available for this dataset.

We can call these data frames directly from `ASRgenomics` using:

```
library(ASRgenomics)
```

Let's explore the first few lines of each of these data frames:

```
head(pheno.salmon)
```

```
##           ID mean_gill_score amoebic_load
## 1 Fish_00001            3.00        28.72
## 2 Fish_00002            3.50        26.47
## 3 Fish_00003            2.00        32.78
## 4 Fish_00004            3.25        26.90
## 5 Fish_00005            3.00        30.34
## 6 Fish_00007            2.50        31.36
```

```
geno.salmon[1:5, 1:5]
```

```
##            AX.87873561 AX.87873586 AX.87873607 AX.87873625 AX.87873669
## Fish_00079           0           2           2           2           2
## Fish_00203           0           1           2           2           2
## Fish_00516           0           2           2           2           1
## Fish_00740           0           1           1           2           2
## Fish_00827           0           2           1           2           2
```

A bit of exploration of the phenotypic data is good practice. First, note that there is a single record for each of the 1,481 individuals, and therefore there is a 1-to-1 match with the genoytpic data (same row dimension). We will discuss later on how to deal with the case of replicated measurements per genotype. The data frame `pheno.salmon` does not contain any other columns relevant as additional factors or covariates that might be required to help control for nuisance effects.

Finally, none of the response variables of interest (`mean_gill_score` and `amoebic_load`) from the phenotypic dataset have missing values. However, if they did, this would not be of concern as these records (and corresponding genomic data) would be dropped from the analyses at the *pre*-GWAS stage.

Let's also explore the genomic data, or **M** matrix. An important consideration is the level of missing data. Here, only 1.1% of the genomic data is missing. When the levels of missing data are less than 5%, a simple mean imputation is recommended. We will implement this using `pre.gwas()` for the present example. For more sophisticated methodologies, there are good external alternatives such as Beagle (Browning et al. 2018) and FImpute (Sargolzaei et al. 2014). Recall that GWAS analyses packages do not normally accept missing data in this **M** matrix, and therefore, often some imputation is performed. However, in `ASRgwas` we can deal with missing data in the **M** matrix using a different (but slower) algorithm that eliminates the need for imputing data providing more reliable results.

We will be using the function `pre.gwas()` to prepare the genomic data for GWAS modelling. But first let's see all the options that it considers.

```
pre.gwas(pheno.data = NULL, indiv = NULL, geno.data = NULL,
    resp = NULL, weights = NULL, map.data = NULL, rename.markers = TRUE,
    Q.method = c("K", "geno.data"), K = NULL, return.K = FALSE,
    message = TRUE, ...)
```

The required inputs are: `pheno.data` that specifies the data frame where all phenotypic data and relevant factors and covariates are; `indiv` and `resp` indicate the names of the columns in `pheno.data` containing the genotype names and the response variable, respectively; and `geno.data` is the matrix with the marker data of form $n \times p$ with individuals in rows and markers in columns. If a data frame with the map is available this can be specified by `map.data`, otherwise a dummy map will be generated.

The above function includes several additional arguments that can be passed, as defined by the '...'; these correspond to filtering options of the **M** matrix, requesting imputation, different methods to obtain the $\mathtt{G}_A$ matrix and to tune it up, etc. These arguments can be listed using the code `pre.gwas()` but they are also listed in the Appendix.

Let's proceed to run our *pre*-GWAS for the Salmon data using the following code:

```
pre.salmon <- pre.gwas(pheno.data = pheno.salmon, indiv = "ID",
    geno.data = geno.salmon, resp = "amoebic_load",
    Q.method = "K", maf = 0.05, marker.callrate = 0.2,
    ind.callrate = 0.2, method = "VanRaden", heterozygosity = 0.8,
    Fis = 0.98, impute = TRUE)
```

This function will execute several operations (output messages are not shown here). First, it will start by filtering the genomic matrix as provided by `geno.data` according to certain specifications, then it will calculate the $\mathtt{G}_A$, or kinship, matrix and evaluate the quality of this matrix. If issues are found in this matrix, then some tune-up will be performed. Diagnostics will be obtained for this matrix to determine if it has some duplicates or issues of concern. Next, a matching of the phenotypic and genomic datasets are implemented to extract only those individuals that are included in both of these data frames. Later, the inverse of the kinship matrix is obtained, and it is assessed to see if it is ill-conditioned. If this happens,

some blending is performed. Finally, the **Q** matrix is obtained based on the kinship matrix unless specified otherwise.

Note that in the above case, we have specified a few additional arguments as detailed below:

- `maf = 0.05` removes markers with minor allele frequency (MAF) below 0.05.
- `marker.callrate = 0.20` removes markers with 20% or more missing values.
- `ind.callrate = 0.20` removes individuals with 20% or more missing values.
- `method = "VanRaden"` requests the calculation of the kinship matrix by the method of VanRaden (2008).
- `heterozygosity = 0.80` removes markers with observed heterozygosity larger than 0.80.
- `Fis = 0.98` removes markers with inbreeding value Fis larger than 0.98.
- `impute = TRUE` requests the use of the average imputation method on the genomic matrix.

The messages from this function report that no individuals were dropped, but 5,980 markers were eliminated for different reasons. Also, only 1.46% of the filtered matrix was imputed. In addition, the diagnostics on the kinship matrix reported no extreme diagonal or duplicates. It was also disclosed that the inverse of this kinship matrix had to be blended once.

From the function `pre.gwas()` many objects are generated as part of the output, and once verified these can be used directly for the GWAS model fit. As seen from running the previous code for the Salmon data, there are several messages and in this case nothing seems out of the ordinary. But for further details we refer to the library `ASRgenomics` (Gezan et al. 2022b) for an explanation of some of these messages.

It is important to explore a little bit of what this function produced by using:

```
ls(pre.salmon)
```

```
## [1] "eigenvalues" "geno.data"   "K"           "Kinv"        "map.data"    "pheno.data"
## [7] "plot.scree"  "Q"
```

And some brief output from here is shown below corresponding to a few lines of the dummy map, the final filtered genomic matrix and the inverse of the kinship matrix.

```
head(pre.salmon$map.data)
```

```
##        marker chrom pos
## 1 AX.87873561     1   1
## 2 AX.87873586     1   2
## 3 AX.87873607     1   3
## 4 AX.87873625     1   4
## 5 AX.87873669     1   5
## 6 AX.87873695     1   6
```

```
pre.salmon$geno.data[1:5, 1:5]
```

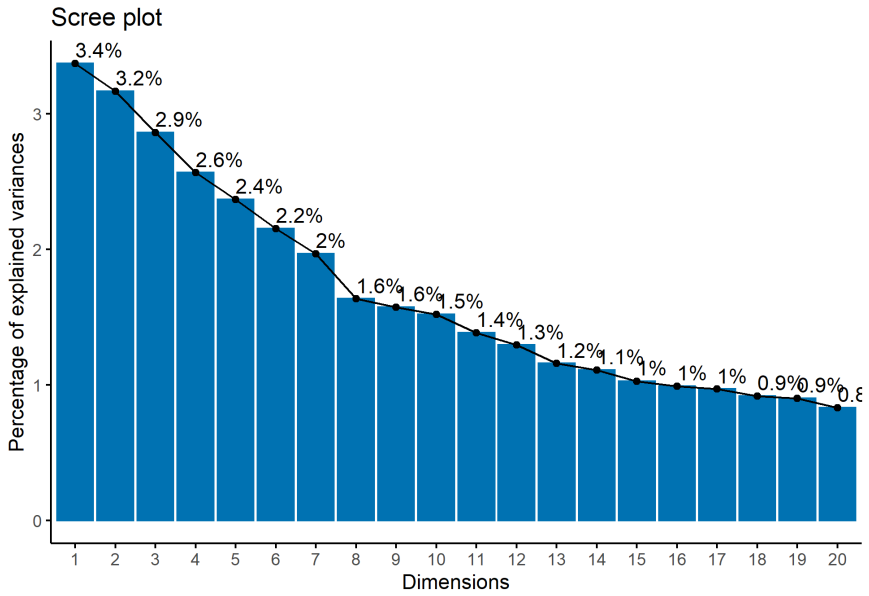
```
##            AX.87873561 AX.87873586 AX.87873607 AX.87873625 AX.87873669
## Fish_00001           0           1           2           2           1
## Fish_00002           1           0           2           1           2
## Fish_00003           1           2           2           1           0
## Fish_00004           0           0           2           2           0
## Fish_00005           0           1           1           2           1
```

```
head(pre.salmon$Kinv$ID)
```

```
##      Row Col     Value
## [1,]   1   1  5.173169
## [2,]   2   1  0.033476
## [3,]   2   2  3.955463
## [4,]   3   1  0.030978
## [5,]   3   2 -0.029644
## [6,]   3   3  5.567874
```

At this moment an important output is the one associated with the **Q** matrix. This is used to help determine the number of dimensions to use to describe the population structure. We will use the scree plot to assess this as shown below

```
pre.salmon$plot.scree
```



From this plot, seven dimensions seem appropriate to describe the structure. Hence, we will consider the parameter `ncp = 7`.

Now, we are finally in a position to run the main function `gwas.asreml()` for the Salmon example:

```
gwasS <- gwas.asreml(pheno.data = pre.salmon$pheno.data,
    resp = "amoebic_load", gen = "ID", Kinv = pre.salmon$Kinv,
    Q = pre.salmon$Q, npc = 7, geno.data = pre.salmon$geno.data,
    map.data = pre.salmon$map, pvalue.thr = 5e-04,
    bonferroni = FALSE, P3D = TRUE)
```

In the above code it is clear that all data frames come from the object `pre.salmon`. Specifically, we are going to perform a GWAS on the response `amoebic_load` and we are indicating that for the **Q** matrix we will use seven dimensions. In addition, we define a p-value threshold of 0.0005. This is somewhat large for this type of analyses but it is used here for illustration purposes. The function `gwas.asreml()` will always consider that a Bonferroni correction is implemented unless `bonferroni = FALSE` is specified. The other arguments are set at their default values.

We have included `P3D = TRUE`, which is the default option where variance components are first estimated without any markers, and then residuals are used to estimate marker effects and their significance using matrix algebra operations. This method is the fastest, but the conventional implementation requires that the genomic matrix has no missing values. If `P3D = FALSE` then the variance components are estimated while fitting each marker. This option allows for missing values in the genomic matrix providing more reliable results, but it is slower.

From running the above code, we have a few messages that report the progress of some of the steps for this GWAS analysis (not shown). The main result for this Salmon dataset is that we have a total of nine markers identified as significant. In addition, we have a false discovery rate (FDR) reported to be 62.1% for these nine markers. Before moving forward, let's explore what objects we have as part of the output from running the above code:

```
ls(gwasS)
```

```
## [1] "aov"          "call"         "gwas.all"     "gwas.sel"     "heritability"
## [6] "mod"
```

One of the first elements to check is the `gwasS$call`. This corresponds to the definition of the base model as fitted in `ASReml-R`. We also have `gwasS$mod`, the complete `asreml` object which we can explore (for example, extract the estimated variance components, generate residual plots, etc.). This is illustrated with the following code:

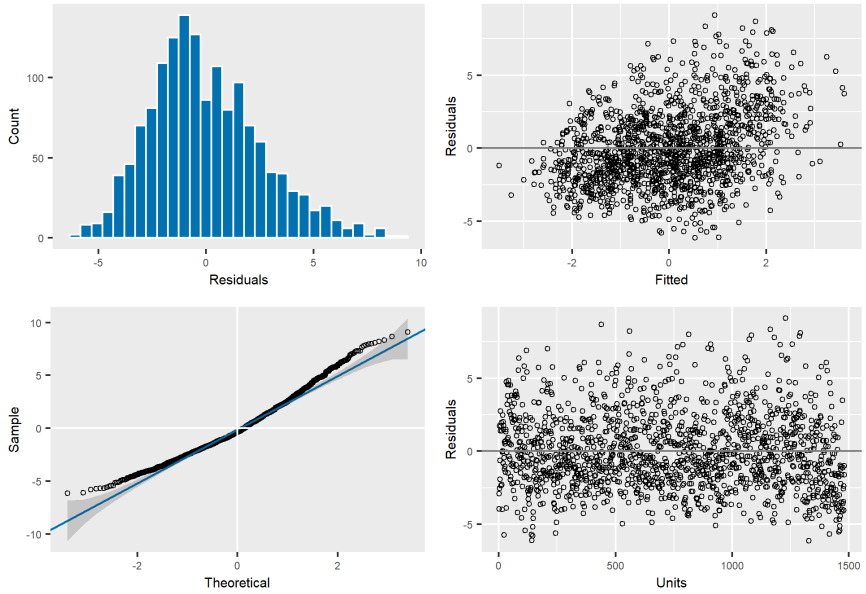
```
gwasS$call
```

```
## asreml::asreml(fixed = amoebic_load ~ 1 + grp(Q), random = ~+vm(ID,
##     pre.salmon$Kinv$ID), residual = ~idv(units), data = pheno.data,
##     na.action = list(x = "include", y = "include"), group = list(Q = c(3,
##         4, 5, 6, 7, 8, 9)), workspace = "1Gb")
```

```
summary(gwasS$mod)$varcomp
```

```
##                               component std.error z.ratio bound %ch
## vm(ID, pre.salmon$Kinv$ID)       2.6436   0.50562  5.2284     P   0
## units!units                      7.8873   0.38013 20.7490     P   0
## units!R                          1.0000        NA      NA     F   0
```

```
plot(gwasS$mod)
```



We can also access other elements, such as the analysis of variance table `gwasS$aov`. However, one of the most important elements is the narrow-sense heritability, as shown below:

```
gwasS$heritability
```

```
## $h2.vc
##                               Estimate       SE            Formula
## vm(ID, pre.salmon$Kinv$ID)  0.25103 0.041127 h2.vc~(V1)/(V1+V2)
##
## $h2.pev
##                               Estimate
## vm(ID, pre.salmon$Kinv$ID)   0.4028
```

There are two expressions of the heritability used here. The expression `h2.vc` is calculated using the estimated variance components from the base model, and the expression `h2.pev` is calculated using the prediction error variances (PEVs) from the BLUP values (for further details see Cullis et al. 2006). Note that the numerator for the heritability is obtained from the variance components associated with the variable specified using the `gen` argument of `gwas.asreml()`.

For the Salmon dataset, both of these expressions are promising estimates, where we expect that a reasonable portion of the phenotypic variability is explained by the genetic components.

Finally, there are two data frames with the marker effects. The first, `gwasS$gwas.all`, reports effects for all markers, whereas the second, `gwasS$gwas.sel`, reports effects for only the significant ones. We are going to explore the latter one below.

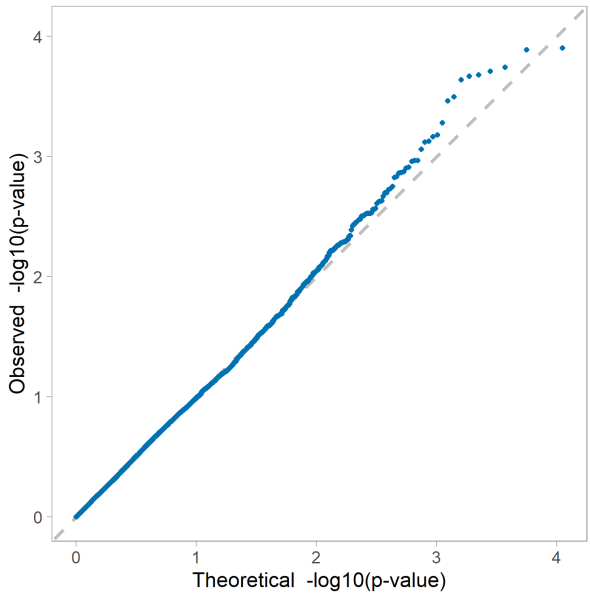
```
gwasS$gwas.sel
```

```
##           marker chrom  pos     maf   effect std.error z.ratio    p.value expl.var
## 554  AX.87895236     1  554 0.29845  0.60781   0.16460  3.6928 0.00022994   1.4704
## 838  AX.87905862     1  838 0.44227  0.64308   0.17226  3.7332 0.00019630   1.9392
## 3031 AX.87993122     1 3031 0.28359  0.71796   0.19358  3.7088 0.00021602   1.9909
## 3135 AX.87997813     1 3135 0.42201  0.60338   0.16238  3.7158 0.00021011   1.6881
## 4928 AX.88069657     1 4928 0.42201 -0.75740   0.21121 -3.5859 0.00034686   2.6600
## 6085 AX.88111674     1 6085 0.37745  0.66234   0.17648  3.7531 0.00018142   1.9596
## 6605 AX.88133783     1 6605 0.32917 -0.69633   0.18147 -3.8371 0.00012977   2.0354
## 8732 AX.88221944     1 8732 0.23768  0.64501   0.16770  3.8463 0.00012506   1.4330
## 9530 AX.88256525     1 9530 0.31668 -0.70917   0.19657 -3.6078 0.00031913   2.0688
```

Note that this is a list of *candidate* markers as they were evaluated individually. The first four columns of the above data frame correspond to information on the specific marker. In this case, we used a dummy map, so all these markers are found on the same chromosome, and their position is only a correlative number. We also have the MAF for reference, followed by estimated effects with their standard errors and corresponding p-values. Finally we have an approximated calculation of the variance explained by each individual marker. Note that these might add more than 100% as markers are evaluated independently.

In `ASRgwas` we can obtain a Q-Q plot and a Manhattan plot based on the complete set of markers. The generation of these plots is very flexible, with several options available within `ASRgwas` to present them as required. For further details we refer you to the help in the library. This time we will only show the Q-Q plot using the code:

```
qq.plot(gwas.table = gwasS$gwas.all)
```



From the Q-Q plot we seem to have an interesting set of markers for this response variable. However, none of them seem to be very strong. We will proceed to evaluate all markers together as a set. As indicated earlier this is done using the function `select.marker()` that will perform a backward selection. This is shown below; however, first we need to extract the marker data from the **M** matrix for the nine selected markers. This is done using the code below where we form the genomic matrix `Msel`, which has dimensions of 1,481 individuals by 9 markers:

```
sel.markers <- gwasS$gwas.sel$marker
Msel <- pre.salmon$geno.data[, sel.markers, drop = FALSE]
```

Now, we can proceed to scan over these nine markers using the code shown below.

```
gwasFIN <- select.marker(gwas.object = gwasS, geno.data.sel = Msel,
    ref.vc = 1, pvalue.thr = 0.01, maxiter.update = 2)
```

In this code we have specified the significance level used to keep markers (`pvalue.thr = 0.01`). In addition, we have indicated the position of the variance component associated with the genetic effects (and kinship matrix) in order to calculate the variance explained for the complete final model (`ref.vc = 1`). Note, the positions of the variance components can be found using `summary(gwasS$mod)$varcomp`. We have indicated that a total of two additional iterations are executed in case one of the models fails to converge (`maxiter.update = 2`). All relevant information and the definition of the base model will be extracted from the main object specified by the `gwas.object`.

This function might take a while to run, especially if there are many pre-selected markers, as the LMM has to be fitted several times.

Before it fits any models, this function will first assess if there are markers with full collinearity (*i.e.*, correlation of 1). If that is the case it will stop and report this set under the object `gwasFIN$collinear.markers`. Otherwise, it will continue eliminating markers until all remaining markers are significant. The list of objects presented as output provide information about the final fitted model (like the `$call` and the full final model in `$mod`). Two important elements are the final table of significant markers and the calculation of the variance explained by these significant markers.

```
ls(gwasFIN)
```

```
## [1] "aov"      "call"     "expl.var" "gwas.sel" "mod"
```

```
gwasFIN$gwas.sel
```

```
##        marker chrom  pos     maf   effect std error z.ratio    p.value expl.var
## 1 AX.87895236     1  554 0.29845  0.54226   0.16838  3.2204 0.00103416  1.17037
## 2 AX.87997813     1 3135 0.42201  0.41467   0.16480  2.5162 0.00928455  0.79733
## 3 AX.88069657     1 4928 0.42201 -0.69262   0.20345 -3.4043 0.00046499  2.22445
## 4 AX.88111674     1 6085 0.37745  0.63334   0.17194  3.6835 0.00014272  1.79183
## 5 AX.88221944     1 8732 0.23768  0.61461   0.16532  3.7177 0.00011109  1.30111
## 6 AX.88256525     1 9530 0.31668 -0.75650   0.19351 -3.9094 0.00007153  2.35425
```

```
gwasFIN$expl.var
```

```
##                             expl.var
## vm(ID, pre.salmon$Kinv$ID)    31.76
```

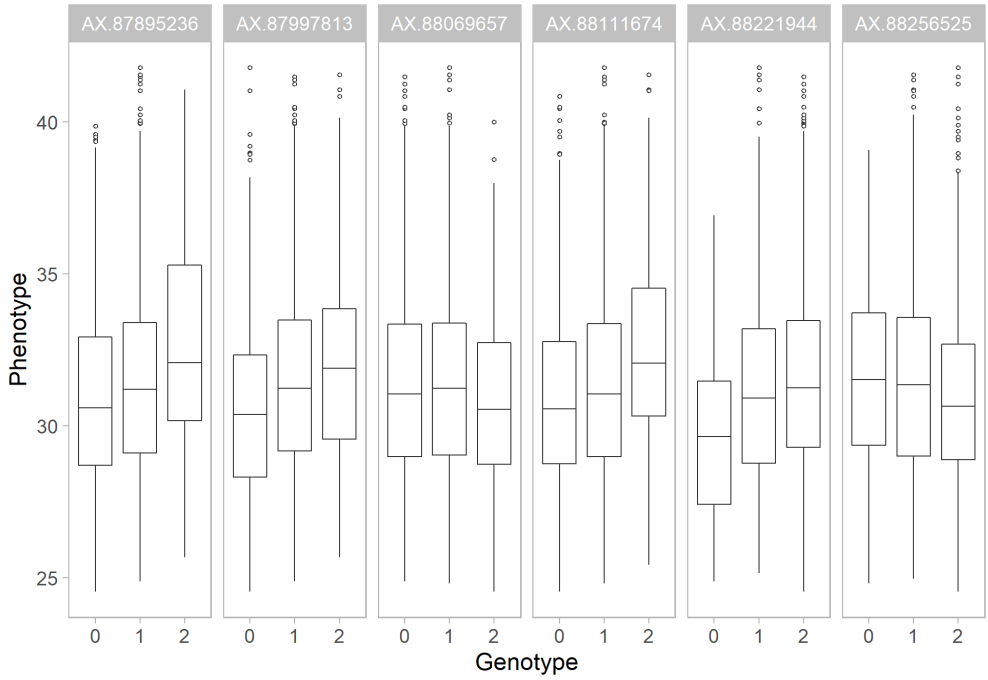
After the backward selection we dropped three markers, leaving the final model with a total of six markers. It is important to define an adequate level of significance for these markers. Some authors suggest using a Bonferroni correction at this stage, others want to maintain the original significance level.

The above function reported that the final model with these six markers explains 31.8% of the genetic variability. This is calculated as the reduction in genetic variance from the base model once these six markers are added. Note that this value will not be equal to the `expl.var` column in the table of marker effects, as this column provides an approximation to explain each marker's contribution. Also, note that the marker effects and their standard errors will have have changed between the initial screening and the final model. This is to be expected as markers have been dropped from the model. The differences, however, are not large in this example.

The final step is to perform some *post*-GWAS. One important thing from the above table is to determine if some of the markers selected are found on the same DNA segment, potentially representing the same gene. This can be checked by looking at the position of the markers or from additional information that is available from understanding the origin of these markers.

We can proceed to obtain a plot with the marker information from the **M** matrix. This is done using the function `marker.plot()`, but before we use it, we need to filter the matrix **M** again to select only those final six markers. The code to do this is

```
Msel <- pre.salmon$geno.data[, gwasFIN$gwas.sel$marker,
    drop = FALSE]
marker.plot(pheno.data = pre.salmon$pheno.data, indiv = "ID",
    resp = "amoebic_load", geno.data.sel = Msel)
```



The above plot is a summary (using boxplots) of the different values of the markers and we can also see better the effect, or phenotypic response, for each markers. For example, the last marker has a negative effect with the phenotypic response tending to decrease with increasing numbers of the major allele. However, the difference between classes 0 and 1 is small.

As with the other plots, there are several options to enhance this output within `ASRgwas` and we refer you to the library's help for additional details.

5 Variants of the GWAS Model Fit

In the previous section we presented a complete example of a GWAS model fit that illustrated the most typical approach used. In the following sections we will present some additional tools and options that allow the model to be fitted differently or in a more flexible way, and also we will present extensions using other analytical approaches.

5.1 Preparing and Auditing data for GWAS

Recall that the workflow of the library `ASRgwas` starts with the `pre.gwas()` function that prepares all data internally for the GWAS model fit. However, it is possible to avoid this step and provide all the required elements, constructed by other routines/libraries or using your own code, to the function `gwas.asreml()` directly. For example, you might already have your genomic matrix $\mathtt{G}_A$ with different marker data and/or tune-up strategies. Or, you may want a different parametrization of your genomic matrix (for example as a dominance matrix). These objects can be provided directly to `gwas.asreml()` but in order to ensure that all attributes and elements are compatible we have included the function `audit.gwas()`. This function will perform a series of checks that will first verify that the phenotypic and genomic data is compatible, and suitable for `gwas.asreml()`. It will also provide some messages and summaries to help implement corrections when this is not the case.

Note that data checks are also thoroughly performed in `gwas.asreml`, but using `audit.gwas()` will give the opportunity to better identify and explore potential issues before forwarding the objects to the final analysis.

For our Salmon example, the code we can use is:

```
check <- audit.gwas(pheno.data = pre.salmon$pheno.data,
    resp = "amoebic_load", indiv = "ID", Kinv = pre.salmon$Kinv,
    geno.data = pre.salmon$geno.data, map.data = pre.salmon$map.data,
    Q = pre.salmon$Q)
```

```
## No mismatches found in the provided data frames.

## A total of 1481 unique genotypes were found.

## A total of 11176 markers were found.

## Individual (genotype) call rate range in 'geno.data' is: 100 ~ 100.

## Marker call rate range in 'geno.data' is: 100 ~ 100.

## Minor allele frequency in 'geno.data' is: 0.05 ~ 0.5.

## Condition of 'Kinv' matrix(ces) is: ID: well-conditioned.
```

As you can see from the messages, this is a good set of objects for use later in `gwas.asreml()`.

5.2 Alternative Analytic Algorithms to Evaluate Markers

The most common and fastest approach to evaluate the significance of the markers is with the algorithm `P3D` as described earlier. Note that here, variance components are estimated only once (in the base model), and then used to test each marker based on an approximated algorithm that uses the residuals from this base model (for further details see Zhang et al. 2010).

In `ASRgwas` we implemented the exact approach that updates the base model by adding each marker individually and re-iterating over the REML variance component estimation. This approach is slower, but it provides exact marker effects and p-values. In addition, this approach allows for missing marker data in the genomic matrix providing more reliable results. This exact approach is implemented when `P3D = FALSE`, but note that running the analysis might take a considerable amount of time especially when the genomic matrix is large.

As an illustration, the `P3D = FALSE` was performed on the same data as before, but without imputation (*i.e.*, `impute = FALSE`) using first the function `pre.gwas()` and then `gwas.asreml()`. Recall that there is only 1.46% missing data in the filtered genomic matrix. The correlations between the SNP effects and the p-values for the earlier analysis, which used the P3D algorithm and imputed the missing data, and this exact analysis were 0.9986 and 0.9952, respectively. Hence, the results are very similar for this particular dataset.

Missing values in the genomic matrix do not allow the use of the fast, but approximated, `P3D` algorithm (unless the missing values are imputed), requiring us to use the much slower exact option. However, a different approximated and fast algorithm was implemented in `ASRgwas` that allows for missing data in the $\mathbf{M}$ matrix. The `P3D` algorithm requires the inverse of the variance-covariance model matrix of fitted values for the base model. However, when there is missing data in a given marker, a new matrix needs to be defined and its inverse obtained. This inverse is approximated iteratively in the function `gwas.asreml()` by using the Schulz's (Petković 2014) or Woodbury's method (Hager 1989). This is indicated with the option `inverse.update`, where a value of -1 requests Woodbury's method and any positive value will perform that number of iterations on the Schulz's method. Our default option is `inverse.update = -1` (*i.e.*, Woodbury's method).

5.3 Variants of the Traditional GWAS model

In our original example we included both the $\mathbf{Q}$ and the $\mathbf{K}$ matrices, by considering the PCA's eigenvectors and the $\mathtt{G}_A$ matrix, respectively. It is possible, and frequently done, to assess different models, for example by dropping the $\mathbf{Q}$ matrix, which is done by using `Q = NULL`. This and other variants can be easily fitted within `gwas.asreml()`. A few of these models are presented in the Appendix.

It is typical in most GWAS to pre-process the phenotypic data by fitting a LMM with the desired structure, and then obtaining a unique *adjusted* response for each genotype. This new response is later used directly in the GWAS model fit as the main response. `ASRgwas()`

works with this *adjusted* response, but it is also flexible enough to use replicated or raw data directly and/or to consider as many fixed and random effects as required. We will illustrate this in the next section with a replicated case and where the above pre-processing step is not necessary.

Nevertheless, if *adjusted* values are going to be used, it is important to consider that there will be a loss of information. In order to minimize this, we recommend to incorporate weights into the model fit, which are approximated as the inverse of the prediction error variance (PEV) (Smith et al. 2001). The function `gwas.asreml()` will accept these weights (through the argument `weights`) and use them to fit the LMM. This is also illustrated in a detailed example in the Appendix.

6 Using Replicated Observations

The library `ASRgwas` is flexible enough to use replicated data directly, and to consider as many fixed effects, random effects, and covariates as required, and with specification of heterogeneous errors. In this example, we will illustrate some of these capabilities.

The dataset used here is from an Apricot (*Prunus armeniaca*) study published by Nsibi et al. (2020). It consists of a F1 generation of 153 individuals from a cross between two cultivars. These were evaluated in southern France during the years 2006 and 2007, but here we focus only on the data from 2006. Several phenotypic measurements are available from fruits collected. For further details we refer you to the original manuscript.

The phenotypic, genotypic and map data for this study is available within `ASRgwas` and it can be accessed by their names.

```
pheno.apricot
geno.apricot
map.apricot
```

The genotypic data is available for all 153 individuals over a total of 60,991 markers, but this dataset has approximatelly 34.4% missing values. In the code below, we present a few of rows of the phenotypic data.

```
head(pheno.apricot)
```

```
##         Ind Years Lots F.Weight Hue.g Ethylene   RI    TA Glucose Fructose Sucrose Citric.A. Malic.A.
## 1 GoMo-A002  2006    1     58.0 73.21     3.18 13.6 20.82    1.60     0.64    5.19     20.01     6.76
## 2 GoMo-A002  2006    2     58.4 71.48     5.04 14.4 19.02    1.65     0.71    6.09     17.87     6.18
## 3 GoMo-A002  2006    3     67.3 70.93     7.81 15.1 17.25    1.47     0.61    6.48     15.07     5.00
## 4 GoMo-A003  2006    1     60.0 70.26     3.40 13.6 22.06    1.98     0.87    6.38     19.15     7.67
## 5 GoMo-A003  2006    2     66.9 68.06     5.59 13.8 22.63    1.98     0.84    6.63     17.95     7.98
## 6 GoMo-A003  2006    3     70.0 68.73     4.42 15.0 20.28    1.97     0.81    8.00     17.24     7.94
```

The dimension of the phenotypic data frame is 474 records, but there is a total of 153 individuals. This is because for each genotype, phenotypic measurements were obtained by collecting the fruit in (mostly) three lots. Hence, we have approximately three replicated measurements per individual. For this reason, we will need to include the factor `Lots` in the GWAS model as a fixed effect to be included in the LMM.

The *pre*-GWAS code is presented below (for which we do not request imputation). The filtered genomic matrix contains only a total of 9,613 markers and this contains 13.4% missing values. In addition, the scree plot suggests that five dimensions should be used for fitting the GWAS model.

```
pre.apr <- pre.gwas(pheno.data = pheno.apricot, indiv = "Ind",
    resp = "Sucrose", geno.data = geno.apricot, map.data = map.apricot,
    maf = 0.05, marker.callrate = 0.4, ind.callrate = 0.4,
    method = "VanRaden", rename.markers = TRUE, heterozygosity = 0.8,
    Fis = 0.98, impute = FALSE, Q.method = "K")
```

The code below fits the GWAS model for the response variable `Sucrose`. Note that the factor `Lots` is included as a fixed effect in `fixedf` and also we are specifying heterogeneous errors by groups also defined by `Lots` by using the `residual` option. The rest is similar to the previous analyses.

```
gwasAR <- gwas.asreml(pheno.data = pre.apr$pheno.data,
    resp = "Sucrose", gen = "Ind", Kinv = pre.apr$Kinv,
    Q = pre.apr$Q, npc = 5, geno.data = pre.apr$geno.data,
    map.data = pre.apr$map.data, pvalue.thr = 5e-04,
    bonferroni = FALSE, fixedf = "Lots", residual = "Lots")
```

The above GWAS model resulted in suggesting three markers as candidates. To illustrate interpretation, let's take the marker `Pp07_13360648_C_G`. This marker, from the table below, has an effect of 0.979. Hence, by considering the nuclotide letters in its name, changing an allele from `C` to `G` will result in an increase of 0.979 in the Sucrose response (corresponding to the units grams of sucrose per 100 grams of fresh weight).

```
gwasAR$gwas.sel
gwasAR$call
summary(gwasAR$mod)$varcomp
```

```
##                 marker chrom      pos     maf   effect std.error z.ratio    p.value expl.var
## 1126 Pp01_28843197_G_C  Pp01 28843197 0.17797 -1.55073   0.44061 -3.5195 0.00048929   40.740
## 8383 Pp07_13360648_C_G  Pp07 13360648 0.23776  0.97891   0.27501  3.5595 0.00041246   20.111
## 8395 Pp07_13708040_C_T  Pp07 13708040 0.27200 -1.16508   0.31432 -3.7067 0.00024116   31.127
```

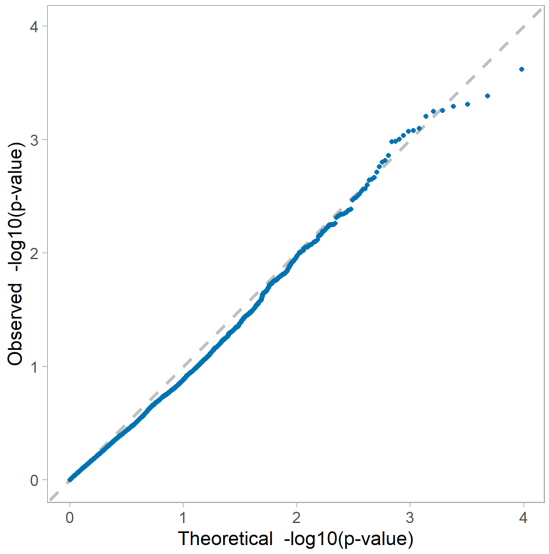
```
## asreml::asreml(fixed = Sucrose ~ 1 + grp(Q) + Lots, random = ~+vm(Ind,
##     pre.apr$Kinv$Ind), residual = ~dsum(~units | Lots), data = pheno.data,
##     na.action = list(x = "include", y = "include"), group = list(Q = c(4,
##         5, 6, 7, 8)), workspace = "1Gb")
```

```
##                             component std.error z.ratio bound %ch
## vm(Ind, pre.apr$Kinv$Ind)   2.11132  0.317366  6.6526     P 0.1
## Lots_1!R                    0.41166  0.070579  5.8326     P 0.0
## Lots_2!R                    0.42393  0.071909  5.8954     P 0.1
## Lots_3!R                    0.82239  0.111380  7.3836     P 0.0
```

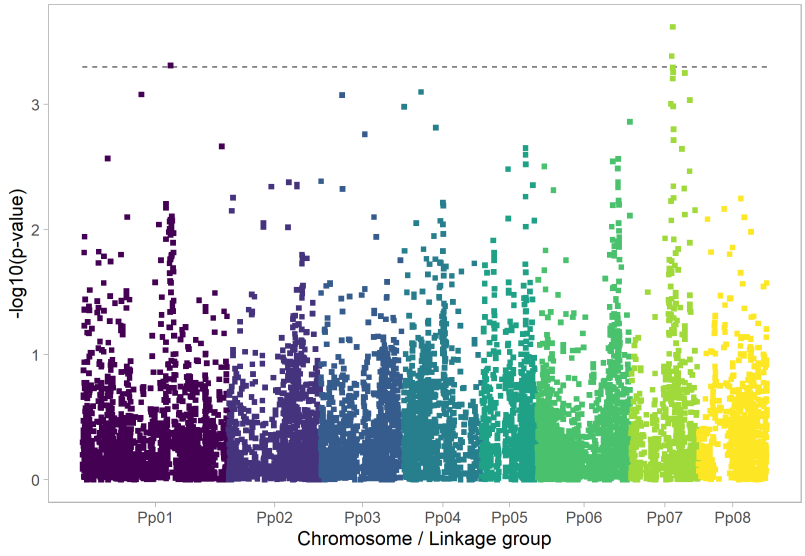
The above output also shows the model call as used by `ASReml-R` where the complexity of the model can be noted, and also variance component estimates are reported. In this case, there are three different residual variances (one for each level of `Lots`), which are not very different.

Finally, we can obtain some additional plots for this analysis, which are presented below. Here, we present not only the Q-Q plot but also the Manhattan plot as generated by `ASRgwas` with a few additional options.

```
qq.plot(gwas.table = gwasAR$gwas.all)
```




```
manhattan.plot(gwas.table = gwasAR$gwas.all, pvalue.thr = 5e-04,
    point.size = 1.2)
```



The above Manhattan plot shows a weak suggestion of the three reported candidate markers affecting the response variable. Interestingly, this weak result is also supported by the calculation of the FDR of 160.22% of the original analysis (not shown here). Hence, it is highly likely that the reported markers are false positives.

7 GWAS for Binomially Distributed Traits

The library `ASRgwas` fits GWAS models with both Normally or Binomially distributed responses. Full modelling flexibility is possible with phenotypic responses that follow a Binomial distribution considering the statistical developments for fitting generalized linear mixed models (GLMM). In this section we will present an example using `ASRgwas` for binary data. The dataset originates from a study that evaluated the viral disease Infectious Pancreatic Necrosis (IPN) on rainbow trout (*Oncorhynchus mykiss*) (Rodriguez et al. 2019). The phenotypic data consists of 749 individuals assessed for resistance traits at the end of a challenged experiment that lasted 63 days. Here, we will focus on the binary trait of IPN mortality coded as 0 if the fish is alive or 1 if it has died at the end of the study. A 36.2% of the fish were reported as dead in this dataset. The raw genotypic data for this study contains a total of 753 individuals with 40,150 SNP markers. Further details can be found in the original reference.

We can load the associated data frames using:

```
library(ASRdata)
```

To explore a few lines of the phenotypic data we use:

```
head(pheno.trout)
```

```
##             ID sex animal_grp contemp_grp tag_wgt_age tag_wgt tag_length
## 1 CD2014000001   1         75 GR00220075I         172     3.2        6.8
## 2 CD2014000002   1         75 GR00220075I         172     2.9        6.5
## 3 CD2014000003   1         75 GR00220075I         172     2.6        5.5
## 4 CD2014000005   1         75 GR00220075I         172     2.6        5.0
## 5 CD2014000006   1         75 GR00220075I         172     2.6        6.0
## 6 CD2014000007   1         75 GR00220075I         172     2.5        5.8
##   ipn_mortality ipn_time_resist end_wgt     family
## 1             0              62      17 CD20140020
## 2             0              62      23 CD20140020
## 3             0              58      22 CD20140020
## 4             0              61      20 CD20140020
## 5             0              62      15 CD20140020
## 6             0              62      20 CD20140020
```

This dataset contains several important columns to use as part of our GWAS model fit. Our response variable for this example is `ipn_mortality`, and the individual fish are identified under `ID`. The columns `tag_wgt`, `sex` and `family` will be used in our model as a covariate, fixed effect and random effect, respectively.

We can now proceed to prepare both the phenotypic and genomic data for the GWAS modelling, using:

```
pre.trout <- pre.gwas(pheno.data = pheno.trout, indiv = "ID",
    geno.data = geno.trout, resp = "ipn_mortality",
    map.data = map.trout[, 1:3], Q.method = "K", maf = 0.05,
    marker.callrate = 0.2, ind.callrate = 0.2, method = "VanRaden",
    heterozygosity = 0.8, Fis = 0.98, impute = TRUE)
```

The specifications of our arguments for this code are not very different from what we have been using before. The function messages report that no individuals were dropped, but now our genomic matrix is reduced to contain 39,618 SNPs and only 0.42% of the missing marker values were imputed. The diagnostics on the kinship matrix reported six possible duplicate genotypes, but the inverse of this matrix seems to not be ill-conditioned, hence, we will proceed with the objects of `pre.gwas()` as they are provided here to run the main function as shown below:

Note: this is a large and complex example. Hence, it is expected to take hours to run.

```
gwasB <- gwas.asreml(pheno.data = pre.trout$pheno.data,
    resp = "ipn_mortality", gen = "ID", Kinv = pre.trout$Kinv,
    Q = pre.trout$Q, npc = 3, cov = "tag_wgt", fixedf = "sex",
    randomf = "family", family = "binomial", geno.data = pre.trout$geno.data,
    map.data = pre.trout$map, pvalue.thr = 5e-04, bonferroni = FALSE,
    P3D = FALSE, threads = 1)
```

Before we look at some of the output, there are important elements to identify from the above code. First, note that we are using three dimensions to describe the structure (`npc = 3`), and we have included additional terms into the model (a covariate, a fixed and a random factor).

Probably, the most important aspect of this model is `family = "binomial"`, that specifies a Binomial distribution for our response `ipn_mortality`. The count used for this data can be specified with the argument `total`, which is `NULL` and it will be considered correctly as 1 by default as this is binary data.

The other argument that we are considering here is `P3D = FALSE`. We are not able to use the P3D approach (which is approximated and fast), because we are within the framework of GLMMs and not LMMs. Finally, for this example, `threads = 1` requests no use of parallelization. This can be modified using larger values if desired but its implementation will depend on the number of CPU cores and characteristics of the ASReml-R license.

This analysis is going to take some time to run given that we are fitting directly a GLMM for each of the markers independently. In addition, we have close to 40,000 markers to evaluate. After running the above code, the main results, given the `pvalue.thr` used of 0.0005, indicate that there is a total of 25 significant markers, corresponding to a FDR of 79.24%.

We will explore only a few objects in detail for this GWAS model fit. First, let's see the call used by the base model as fitted in ASReml-R, followed by its calculation of the heritability.

```
gwasB$call
```

```
## asreml::asreml(fixed = ipn_mortality ~ 1 + grp(Q) + tag_wgt +
##     sex, random = ~+vm(ID, pre.trout$Kinv$ID) + family,
##     residual = ~id(units), data = pheno.data, na.action = list(x = "include",
##     y = "include"), family = asreml::asr_binomial(link = "logit",
##     dispersion = 1), group = list(Q = c(6, 7, 8)), workspace = "1Gb")
```

```
gwasB$heritability
```

```
## $h2.vc
## NULL
##
## $h2.pev
##                           Estimate
## vm(ID, pre.trout$Kinv$ID) 0.13411
```

The reported heritability seems reasonable, but because we are using binary data, this calculation does not have the same interpretation as with Normally distributed data.

Finally, we can see the object `gwasB$gwas.sel` with only the significant markers. This data frame is similar to the others reported before; however, the effects in this case are associated with the logit link used to fit the GLMM.

```
head(gwasB$gwas.sel)
```

```
##                marker chrom   pos     maf   effect std.error z.ratio    p.value
## 1197  AX_89957186_A_C     1  1197 0.13636  1.18132   0.31598  3.7387 0.00018501
## 8038  AX_89960431_G_T     1  8038 0.13433  1.03614   0.29333  3.5323 0.00041190
## 12185 AX_89933556_G_A     1 12185 0.39959 -0.64721   0.18555 -3.4880 0.00048656
## 12249 AX_89936021_A_G     1 12249 0.46879  0.69044   0.18934  3.6466 0.00026577
## 12615 AX_89948567_C_T     1 12615 0.13704 -0.94079   0.25808 -3.6453 0.00026707
## 12616 AX_89948613_A_C     1 12616 0.13094  1.14428   0.29649  3.8595 0.00011363
```

As indicated before, this is a group of candidate markers, and further exploration is required by using backward selection. In addition, as shown earlier, we can obtain different plots for this data and the GWAS model fit.

8 Closing Remarks

Our aim with `ASRgwas` is to provide a complete tool to implement Genome-Wide Association Studies (GWAS) using the full modelling flexibility as available in `ASReml-R`. We have made efforts to use the latest algorithms in order to have fast calculations and reliable estimates of the marker effects.

The main unique aspect of `ASRgwas`, besides its ability to consider any number of fixed and random effects, is the ability to handle genomic matrices with missing values, therefore avoiding the need for marker imputation. The implemented algorithms are not only available for the analysis of Normally distributed data but also for Binomial data within the context of Generalized Linear Mixed Models (GLMMs). Therefore, `ASRgwas` allows the use of appropriate statistical tools for GWAS on a broader set of traits and cases.

This package is in no way comprehensive, but we have laid the foundation with open and flexible routines that should allow advanced users to test and evaluate more complex, and often more realistic, GWAS models. We consider this library as a dynamic source that in future versions will be expanded to incorporate new functionalities and even faster implementations.

Appendix

Additional Arguments for `pre.gwas()`

The following is a list of additional arguments (and brief descriptions) that can be passed from the function `pre.gwas()` as defined by the '`...`' to the functions available in the library `ASRgenomics`.

Argument	Description
na.string	A character that represents missing values (default = "NA").
marker.callrate	Marker call rate threshold (default = 0.2).
ind.callrate	Individual call rate threshold (default = 0.2).
maf	Minor allele frequency (MAF) threshold (default = 0.05).
heterozygosity	Heterozygosity threshold (default = 1, i.e. no markers removed).
Fis	Fis threshold (default = 1, i.e. no markers removed).
impute	If TRUE imputation of missing values is performed (default = FALSE).
method	Method of genomic relationship matrix (GRM) calculation ("VanRaden", "Yang", "Su", "Vitezica") (default = "VanRaden").
criteria	Criteria ("callrate" or "maf") to choose marker to drop in pruning (default = "callrate").
pruning.thr	Correlation threshold to identify redundant markers for pruning (default = NULL; i.e. no pruning).
by.chrom	If TRUE the pruning is performed independently by chromosome (default = FALSE).
window.n	Number of markers to consider in each window to perform pruning (default = 50).
overlap.n	Number of markers to overlap between consecutive windows when pruning (default = 5).
iterations	Number of sequential times the pruning procedure should be executed (default = 10).
seed	A seed for reproducibility in pruning (default = NULL).
A	A pedigree relationship matrix for blending/alignment (default = NULL).
blend	If TRUE a blending of K is performed (default = FALSE).
pblend	The proportion of pedigree or identity to use in blending (default = NULL).
bend	If TRUE a bending is performed (default = FALSE).
eig.tol	Determines which threshold of eigenvalues will be treated as zero (default = 1e-06).
align	If TRUE the GRM is aligned to the matching pedigree matrix (default = FALSE).
diagonal.thr.large	A threshold value to flag large diagonal values (default = 1.8).
diagonal.thr.small	A threshold value to flag small diagonal values (default = 0.2).
duplicate.thr	A threshold value to flag possible duplicates (default = 0.95).
clean.diagonal	If TRUE filters values < diagonal.thr.large and > diagonal.thr.small (default = TRUE).
clean.duplicate	If TRUE returns a kinship matrix without duplicate individuals (default = TRUE).
rcn.thr	A threshold for identifying the K matrix as an ill-conditioned matrix (default = 1e-12).
scale	If TRUE scales the kinship matrix (default = TRUE).
ncp	The number of principal components (PC) dimensions to provide in the output data frame (default = 20).

Example with Additive and Dominant Effects

In this example we will use the Salmon data presented before. Here, we will process the data frames for both additive and dominance elements. In the code below, we obtain the filtered genomic matrix **Ma** and the genomic relationship matrix $\mathtt{G}_A$ only for additive effects (`method = "VanRaden"`). All of this will be part of the object `pre.salmonA`.

```
pre.salmonA <- pre.gwas(pheno.data = pheno.salmon,
    indiv = "ID", resp = "amoebic_load", geno.data = geno.salmon,
    Q.method = "K", maf = 0.05, marker.callrate = 0.2,
    ind.callrate = 0.2, method = "VanRaden", heterozygosity = 0.8,
    Fis = 0.98, impute = TRUE)
```

Below, we proceed to obtain the same elements but for dominance effects as part of the object `pre.salmonD`. That is, the filtered genomic matrix **Md** and the genomic relationship matrix $\mathtt{G}_D$ (`method = "Su"`).

```
pre.salmonD <- pre.gwas(pheno.data = pheno.salmon,
    indiv = "ID", resp = "amoebic_load", geno.data = geno.salmon,
    Q.method = "K", maf = 0.05, marker.callrate = 0.2,
    ind.callrate = 0.2, method = "Su", heterozygosity = 0.8,
    Fis = 0.98, impute = TRUE)
```

Now, we will process the phenotypic data, but for this, it is necessary to create a factor for the additive effects (`IDA`) and another for dominance effects (`IDD`).

```
pheno.AD <- pre.salmon$pheno.data
pheno.AD$IDA <- pheno.AD$ID
pheno.AD$IDD <- pheno.AD$ID
```

Next, we need to combine the genomic matrices for additive and dominance marker effects; however, the dominance matrix needs to be re-parametrized where all homozygous will have the value of 0, and heterozygous a value of 1. In addition, marker names for the dominance matrix are recoded by adding `D` in front of them.

```
Ma <- round(pre.salmonA$geno.data, 0)
Md <- round(pre.salmonD$geno.data, 0)
colnames(Md) <- paste("D", colnames(Md), sep = "_")
Md[Md == 2] <- 0
Mad <- cbind(Ma, Md)
```

Now we are ready to fit the GWAS model with additive and dominance marker effects. The respective code is shown below. Note the use of the combined names for `gen` using `c()` to allow for both the additive and dominance effect. Also note the list of kinship matrices supplied `Kinv`.

```
gwasS <- gwas.asreml(pheno.data = pheno.AD, resp = "amoebic_load",
    gen = c("IDA", "IDD"), Kinv = list(IDA = pre.salmonA$Kinv$ID,
        IDD = pre.salmonD$Kinv$ID), Q = pre.salmonA$Q,
    npc = 7, geno.data = Mad, map.data = NULL, pvalue.thr = 5e-04,
    bonferroni = FALSE, P3D = TRUE)
```

Some output is presented below, where one of the most relevant aspects is the presence of a narrow-sense heritability of 0.22 and a dominance ratio of 0.04. This indicates that there is little dominance variance explained by the markers. However, there is a total of 14 markers reported as significant, from which five are from the dominance genomic matrix.

```
summary(gwasS$mod)$varcomp
gwasS$heritability
gwasS$gwas.sel
```

```
##                                                                                component std.error  z.ratio bound %ch
## vm(IDA, list(IDA = pre.salmonA$Kinv$ID, IDD = pre.salmonD$Kinv$ID)$IDA)   2.34900   0.61428  3.82401     P 0.2
## vm(IDD, list(IDA = pre.salmonA$Kinv$ID, IDD = pre.salmonD$Kinv$ID)$IDD)   0.44237   0.60423  0.73213     P 1.6
## units!units                                                                7.69571   0.46246 16.64065     P 0.0
## units!R                                                                    1.00000        NA       NA     F 0.0


## $h2.vc
##                                                                                Estimate       SE                  Formula
## vm(IDA, list(IDA = pre.salmonA$Kinv$ID, IDD = pre.salmonD$Kinv$ID)$IDA) 0.223990 0.053441    h2.vc~(V1)/(V1+V2+V3)
## vm(IDD, list(IDA = pre.salmonA$Kinv$ID, IDD = pre.salmonD$Kinv$ID)$IDD) 0.042183 0.057717    h2.vc~(V2)/(V2+V1+V3)
## joint                                                                   0.266173 0.047094 h2.vc~(V1+V2)/(V1+V2+V3)
##
## $h2.pev
##                                                                                Estimate
## vm(IDA, list(IDA = pre.salmonA$Kinv$ID, IDD = pre.salmonD$Kinv$ID)$IDA) 0.366720
## vm(IDD, list(IDA = pre.salmonA$Kinv$ID, IDD = pre.salmonD$Kinv$ID)$IDD) 0.043061


##                   marker chrom   pos     maf   effect std.error z.ratio    p.value expl.var
## 554           AX.87895236     1   554 0.29845  0.59414   0.16431  3.6160 0.00030925   1.4051
## 838           AX.87905862     1   838 0.44227  0.64291   0.17136  3.7518 0.00018236   1.9382
## 3031          AX.87993122     1  3031 0.28359  0.71653   0.19242  3.7237 0.00020371   1.9829
## 3135          AX.87997813     1  3135 0.42201  0.61047   0.16198  3.7689 0.00017043   1.7281
## 4928          AX.88069657     1  4928 0.42201 -0.75869   0.20962 -3.6193 0.00030540   2.6691
## 6085          AX.88111674     1  6085 0.37745  0.66664   0.17596  3.7886 0.00015760   1.9852
## 6605          AX.88133783     1  6605 0.32917 -0.69038   0.18043 -3.8263 0.00013550   2.0007
## 8732          AX.88221944     1  8732 0.23768  0.64169   0.16787  3.8226 0.00013753   1.4183
## 9530          AX.88256525     1  9530 0.31668 -0.71296   0.19489 -3.6583 0.00026282   2.0910
## 13629       D_AX.87969156     1 13629 0.19480 -0.75673   0.20295 -3.7287 0.00019977   1.7075
## 18950       D_AX.88182114     1 18950 0.23396  0.68853   0.17822  3.8633 0.00011671   1.6152
## 20807       D_AX.88261029     1 20807 0.11850 -0.92656   0.25207 -3.6757 0.00024568   1.7048
## 22212 D_Crit2_rs159403402     1 22212 0.24848  0.59853   0.16973  3.5263 0.00043429   1.2717
## 22323  D_Crit_rs159403402     1 22323 0.24713  0.61801   0.16957  3.6447 0.00027706   1.3509
```

Further exploration of the above results can be done with the use of Q-Q and Manhattan plots. In addition, it is possible to proceed to select the relevant markers using the function `select.marker()` to obtain the final list. We leave these elements to the reader.

Example that Incorporates Weights

In this example we will use the raw Apricot data from year 2006 to fit a GWAS model that considers weights. These weights are obtained in the first step of this process using the library `ASRtriala` (Gezan et al. 2022a) together with the genotype's predictions (BLUEs). In the second step, this summarized information is used to fit the GWAS model.

Then, in the following code we obtain the weights for the model using the library `ASRtriala`. Further details of its use can be found in the manual of that library.

Note that we are considering the factor `Lots` as a fixed effect of replication. For clarification, we are presenting the model call used within `ASRtriala` for this code, and we show a few lines of the prediction table with its corresponding weights.

```
library(ASRtriala)
mod06 <- fit.single(data = pheno.apricot, gen = "Ind",
    rep = "Lots", resp = "Sucrose", type.gen = "random",
    type.rep = "fixed", add.rep = TRUE, type.residual = "indep",
    fix.vc = TRUE)
mod06$call
head(mod06$predictions)
```

```
## asreml::asreml(fixed = Sucrose ~ 1 + Lots + Ind, residual = ~idv(units),
##     G.param = vc.table, R.param = vc.table, data = data, na.action = list(x = "include",
##         y = "include"))
```

```
##         Ind predicted.value std.error    status weight
## 1 GoMo-A002          5.9200   0.42189 Estimable 5.6182
## 2 GoMo-A003          7.0033   0.42189 Estimable 5.6182
## 3 GoMo-A004          8.5433   0.42189 Estimable 5.6182
## 4 GoMo-A005          8.0267   0.42189 Estimable 5.6182
## 5 GoMo-A006          9.0467   0.42189 Estimable 5.6182
## 6 GoMo-A007          9.6400   0.42189 Estimable 5.6182
```

Now we can proceed to perform the *pre*-GWAS using the available genomic data frames and the predictions from the above table.

```
pre.apr <- pre.gwas(pheno.data = mod06$predictions,
    indiv = "Ind", resp = "predicted.value", weights = "weight",
    geno.data = geno.apricot, map.data = map.apricot,
    Q.method = "K", maf = 0.05, marker.callrate = 0.4,
    ind.callrate = 0.4, method = "VanRaden", rename.markers = TRUE,
    heterozygosity = 0.8, Fis = 0.98, impute = FALSE)
```

Note in the code above we used `weights = "weight"` to make sure the column of weights is saved and transferred to the output objects from `pre.gwas()`. Finally, we fit the GWAS model but this time with weights. The key aspect again is the use of the option `weights = "weight"` within `gwas.asreml()`.

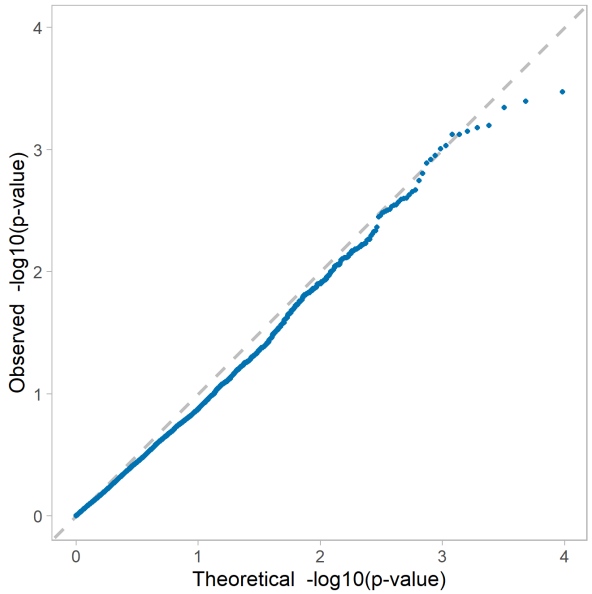
```
gwasA <- gwas.asreml(pheno.data = pre.apr$pheno.data,
    resp = "predicted.value", gen = "Ind", Kinv = pre.apr$Kinv,
    Q = pre.apr$Q, npc = 5, geno.data = pre.apr$geno.data,
    map.data = pre.apr$map.data, pvalue.thr = 5e-04,
    bonferroni = FALSE, weights = "weight")
```

Some results are presented below. These are similar to the ones reported earlier for the replicated dataset, where there in both cases three candidate markers reported, but these are not exactly the same. These differences are moderate, and more unbalanced data or complex residual structures, such as autoregressive errors, will lead to very different conclusions.

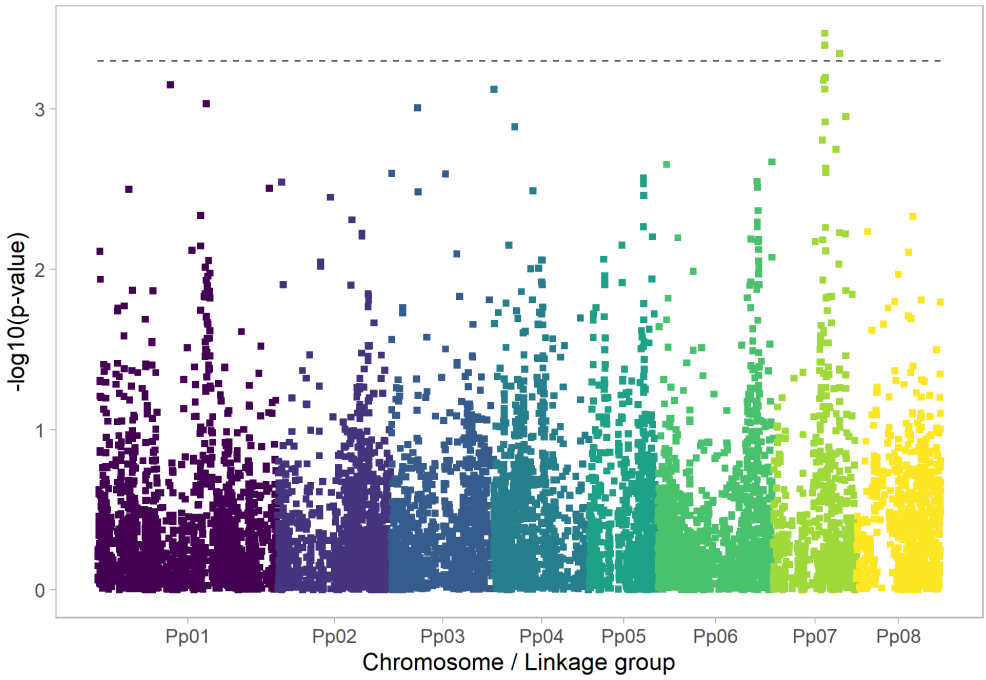
```
gwasA$gwas.sel
```

```
##                marker chrom      pos     maf  effect std.error z.ratio    p.value expl.var
## 8394 Pp07_13697066_G_A  Pp07 13697066 0.26174  1.0780   0.29741  3.6247 0.00040220   43.301
## 8395 Pp07_13708040_C_T  Pp07 13708040 0.27200 -1.1943   0.32351 -3.6917 0.00033872   54.457
## 8620 Pp07_17619560_C_T  Pp07 17619560 0.36577 -0.8316   0.23156 -3.5913 0.00045258   30.933
```

```
qq.plot(gwas.table = gwasA$gwas.all)
```




```
manhattan.plot(gwas.table = gwasA$gwas.all, pvalue.thr = 5e-04,
    point.size = 1.2)
```


-log10(p-value)
3
2
1
0
Pp01
Pp02
Pp03
Pp04
Pp05
Pp06
Pp07
Pp08
Chromosome / Linkage group

Example Illustrating Complex Graphical Output

All of the graphical functions within `ASRgwas` have many options to produce graphs with more aesthetics and additional features. In the following examples we will illustrate this with the functions `qq.plot()` and `manhattan.plot()`. For this, let's assume we haven't yet selected a final GWAS model, hence, we need to check a few Q-Q plots to see which model fit presents better behavior.

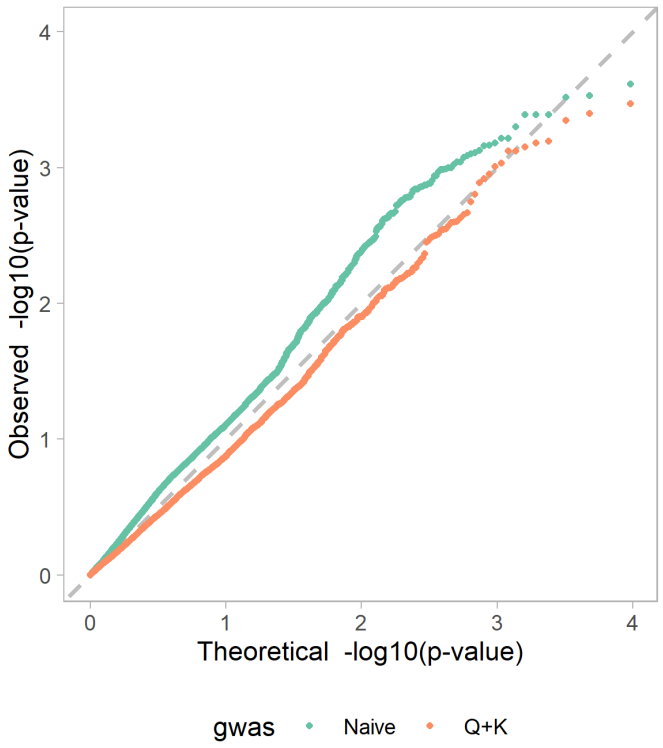
In this example we will compare the model `gwasA` previously shown against a "naive" (marker only) model (code not shown for this one). Consider that the full data frame of marker effects are in objects `gwasA$gwas.all` and `gwasA_naive$gwas.all`, respectively.

Let's first add an indexing variable to these two GWAS results, and then combine them into a single data frame. This is shown in the code below:

```
gwasA$gwas.all$gwas <- "Q+K"
gwasA_naive$gwas.all$gwas <- "Naive"
gwasBoth <- rbind(gwasA$gwas.all, gwasA_naive$gwas.all)
```

Now we request the Q-Q plots making reference to the indexing variable using the code below. Note that additional features are added using an instruction from `ggplot2`.

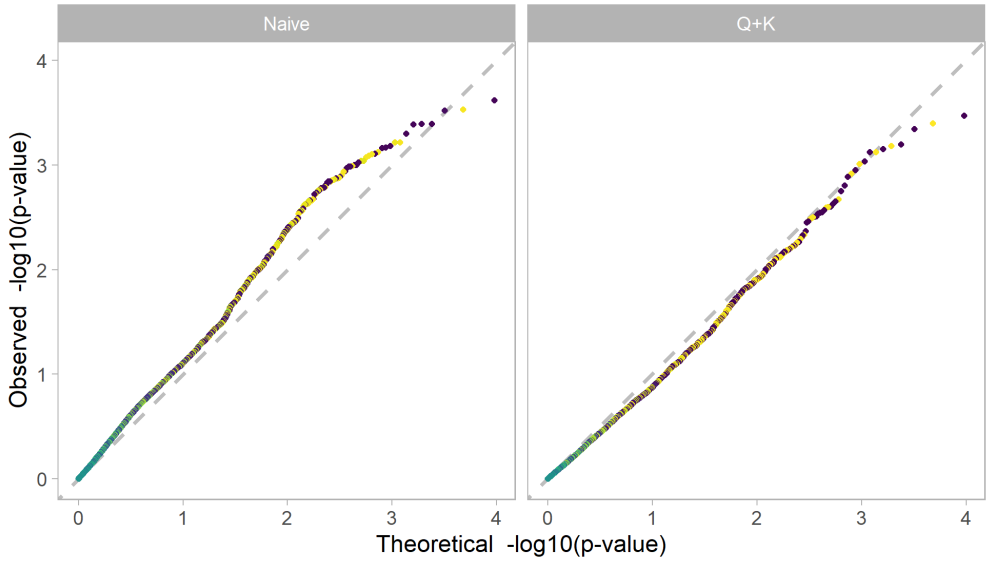
```
qq.plot(gwas.table = gwasBoth, gwas.index = "gwas",
    point.colour = "gwas", legend.position = "bottom") +
    ggplot2::scale_colour_brewer(palette = "Set2")
```



The above plot for the naive GWAS fit suggests the presence of some missing elements in this model, which were considered in the original analyses with the incorporation of the **Q** and **K** matrices, and this is clearly identified by the dots aligned close to the 1-to-1 line.

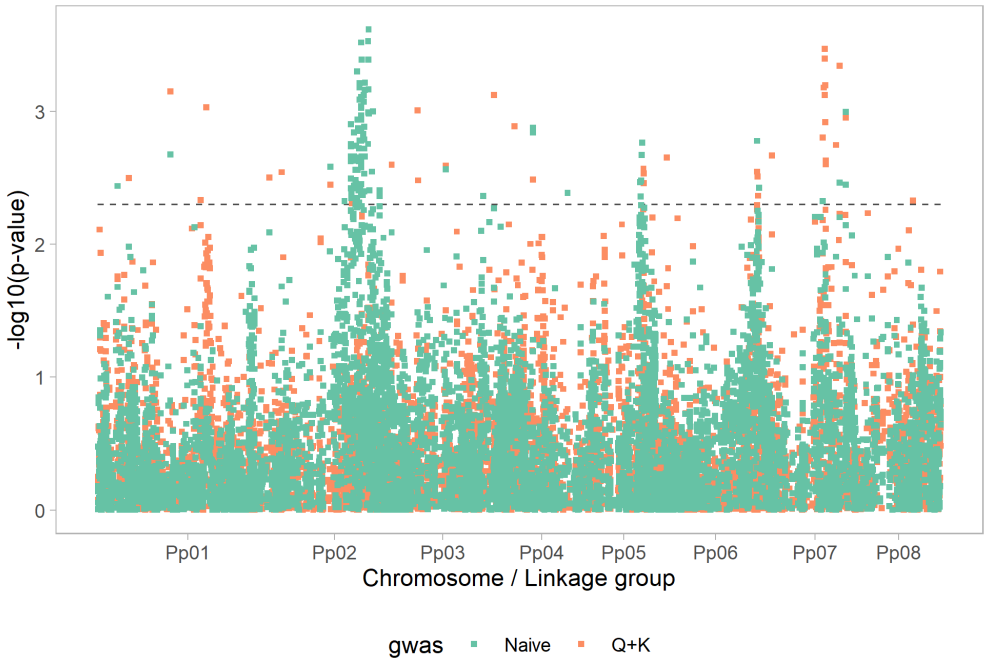
The function `qq.plot()` has several other options to produce additional features. In the example below, a different graph is presented for each of the GWAS model fits, and the colours of each marker is associated with the magnitude of its respective effect (*i.e.*, column `effect`).

```
qq.plot(gwas.table = gwasBoth, gwas.index = "gwas",
    point.colour = "effect", facet.main = "gwas")
```



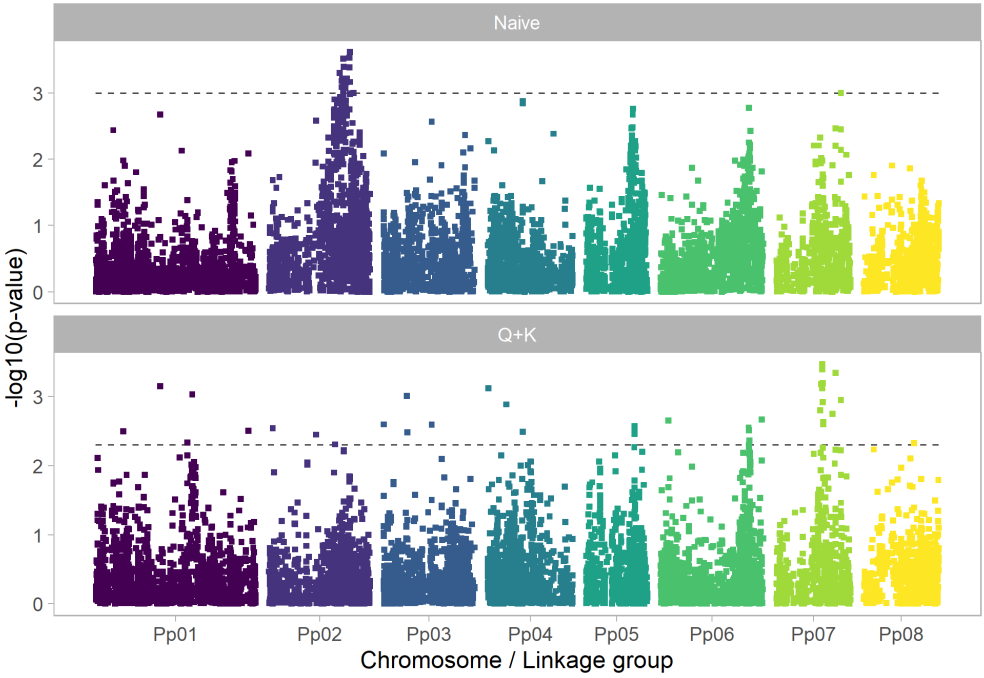
For the Manhattan plot, the logic of the function `manhattan.plot()` is very similar. In the code below, we plot two GWAS model fits on the same panel.

```
manhattan.plot(gwas.table = gwasBoth, pvalue.thr = 0.005,
    gwas.index = "gwas", point.colour = "gwas", legend.position = "bottom") +
    ggplot2::scale_colour_brewer(palette = "Set2")
```



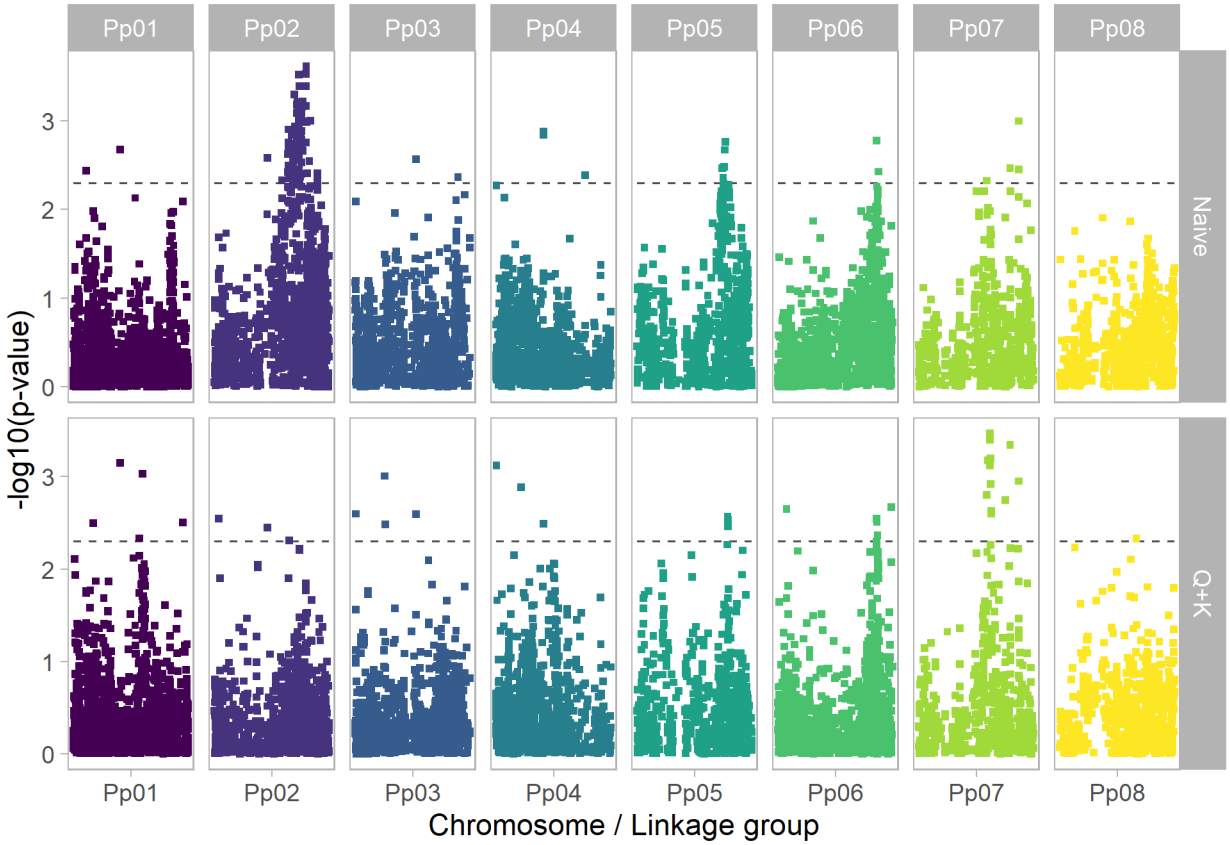
As you can see, the above plot is not as informative as the previous Q-Q plots obtained. But this figure can be greatly improved by: 1) separating the analyses into two vertical panels, 2) allowing for some separation of the chromosomes, 3) adjusting colours, and by adding threshold lines.

```
manhattan.plot(gwas.table = gwasBoth, pvalue.thr = c(0.005,
    0.001), gwas.index = "gwas", collate = TRUE, padding = 4e+06,
    facet.main = "gwas", facet.dim = c(2, 1))
```



Furthermore, we can expand the panels over the x- and y-axis by using the arguments `facet.main` and `facet.secondary`. In this case, chromosome is used as an index, but other elements could be considered, such as trait or any other variable of interest.

```
manhattan.plot(gwas.table = gwasBoth, pvalue.thr = 0.005,
    gwas.index = "gwas", collate = TRUE, padding = 4e+06,
    facet.main = "chrom", facet.secondary = "gwas",
    scales = "free")
```



We have shown only a few of the capabilities and additional attributes available for these plots. For more details we suggest checking out the package help using `help(ASRgwas)`.

Bibliography

Browning, B., Y. Zhou, and S. Browning. 2018. A one-penny imputed genome from next generation reference panels. *Am J Hum Genet*. 103(3):338–348.

Butler, D., B. R. Cullis, A. Gilmour, and B. Gogel. 2009. ASReml-R reference manual. *The State of Queensland, Department of Primary Industries and Fisheries, Brisbane*.

Cullis, B., A. Smith, and N. Coombes. 2006. On the design of early generation variety trials with correlated data. *Journal of Agricultural, Biological, and Environmental Statistics*. 11(4):381–393.

Gezan, S., G. Galli, and D. Murray. 2022a. ASRtriala: An R package with complementary functions for single and multi-environment trial analyses. Version 1.0.0. *VSN International Ltd., Hemel Hempstead, United Kingdom*.

Gezan, S., A. de Oliveira, G. Galli, and D. Murray. 2022b. ASRgenomics: An R package with complementary genomic functions. Version 1.1.0. *VSN International Ltd., Hemel Hempstead, United Kingdom*.

Hager, W. 1989. Updating the inverse of a matrix. *SIAM Review*. 31(2):221–239.

Nsibi, M., B. Gouble, S. Bureau, T. Flutre, C. Sauvage, J. Audergon, and J. Regnard. 2020. Adoption and optimization of genomic selection to sustain breeding for Apricot fruit quality. *G3 Genes, Genomes, Genetics*. 10(12):4513–4529.

Patterson, N., A. Price, and D. Reich. 2006. Population structure and eigenanalysis. *PLoS Genetics*. 2(12):e110.

Petković, M. 2014. Generalized Schultz iterative methods for the computation of outer inverses. *Computers & Mathematics with Applications*. 67(10):1837–1847.

Robledo, D., O. Matka, A. Hamilton, and R. D. Houston. 2018. Genome-wide association and genomic selection for resistance to amoebic gill disease in Atlantic salmon. *G3: Genes, Genomes, Genetics*. 8:1195–1203.

Rodriguez, F., R. Flores-Mare, G. Yishida, A. Barria, A. Jedlicki, J. Lhorente, F. Reyes-Lopes, and J. Yanez. 2019. Genome-wide association analysis for resistance to infectious pancreatic necrosis virus identifies candidate genes involved in viral replication and immune response in rainbow trout (Oncorhynchus mykiss). *G3 Genes, Genomes, Genetics*. 9:2897–2904.

Sargolzaei, M., C. JP, and S. FS. 2014. A new approach for efficient genotype imputation using information from relatives. *BMC Genomics*. 15:478.

Smith, A., B. R. Cullis, and A. R. Gilmour. 2001. The analysis of crop variety evaluation data in Australia. *Australian & New Zealand Journal of Statistics*. 43(2):129–145.

Su, G., O. F. Christensen, T. Ostersen, M. Henryin, and M. S. Lund. 2012. Estimating additive and non-additive genetic variances and predicting genetic merits using genome-wide dense single nucleotide polymorphism markers. *PloS One*. 7(9):e45293.

VanRaden, P. 2008. Efficient methods to compute genomic predictions. *Journal of Dairy Science*. 91(11):4414–4423.

Vitezica, Z. G., L. Varona, and A. Legarra. 2013. On the additive and dominant variance and covariance of individuals within the genomic selection scope. *Genetics*. 195(4):1223–1230.

Yang, J. et. al. 2010. Common SNPs explain a large proportion of the heritability for human height. *Nature Genetics*. 42(7):565–569.

Zhang, Z., E. Ersoz, C.-Q. Lai, R. Todhunter, H. Tiwari, M. Gore, P. Bradbury, et al.

2010. Mixed linear model approach adapted for genome-wide association studies. *Nature Genetics*. 42(4):355–360.